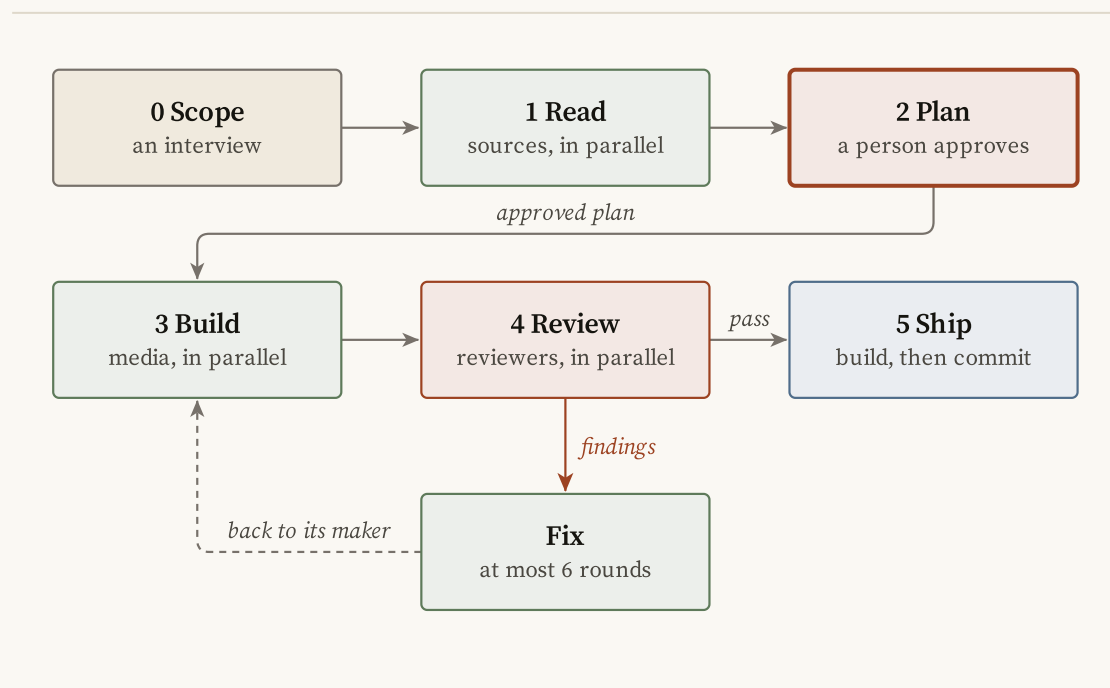


Lessons with a tutor inside

How a team of agents turns course material into a lesson a student can talk to

lesson-builder is a Claude Code skill. Point it at a course's slides, notes and past problems and it builds a lesson: a small web app with the topics in order, the equations typeset, graphs the reader can move, animations, demos, and a tutor panel down the side that knows what the page says. It works in six phases, and agents it spawns do most of the work. Every number here was read off the skill's repo, or the project that uses it, on 23 September 2026, and the grey line at the foot of each page says where.

FIGURE 1 *six phases, and one of them waits for a person*



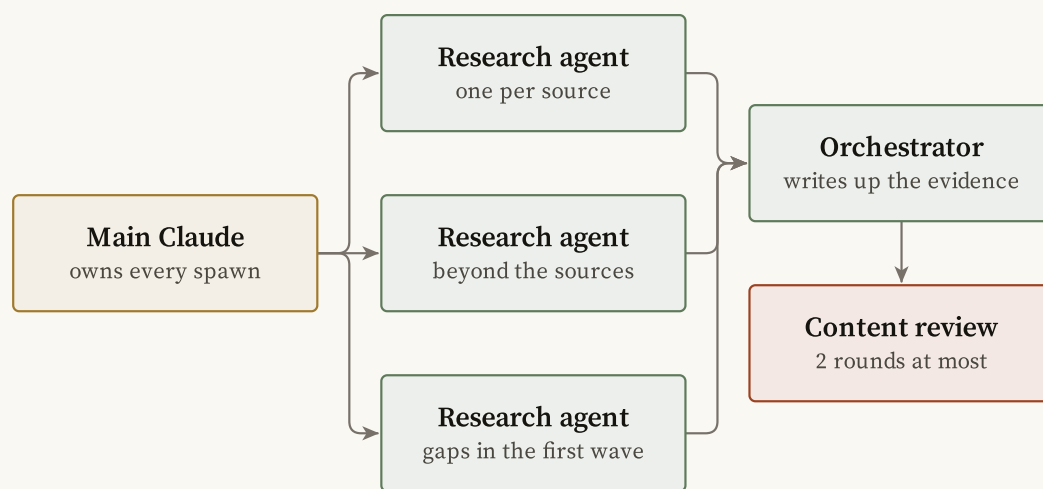
Sage is agents at work, rust a check that can say no, slate what ships. The heavy box is the one place a person decides.

Reading and planning

The first three phases decide what the lesson teaches, and a person signs it off.

Phase 0 is an interview: which materials count, how far past them to go, and where it ships. A build with no one watching answers from the course's notes and safe defaults, and lists what it assumed in the plan.

FIGURE 2 *phase 1 reads every source at once*



Ochre is the session, sage the agents that work, rust the one that checks them.

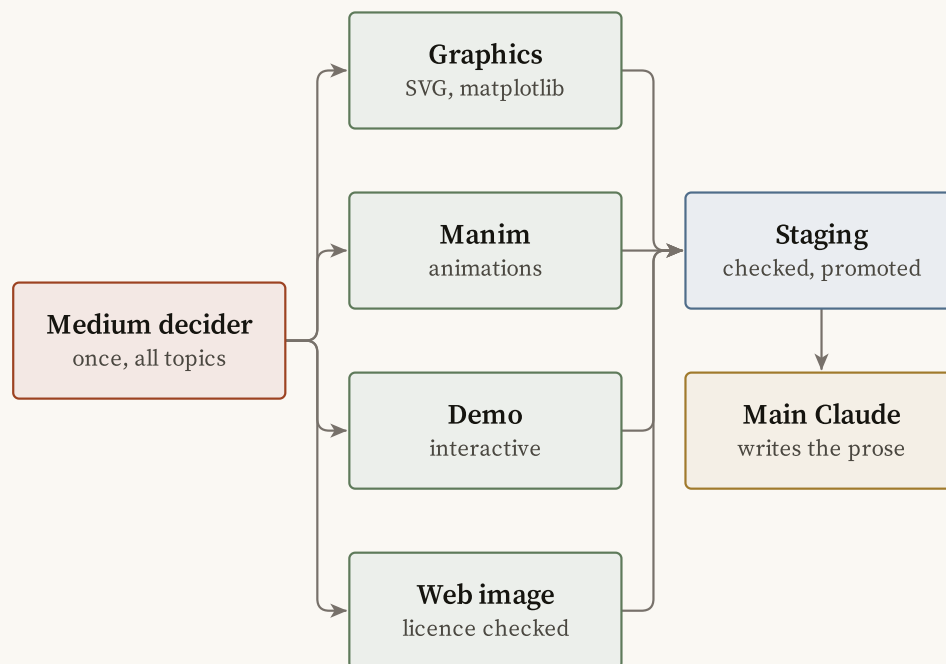
Phase 1 reads. Main Claude spawns one research agent per source, more for ground beyond the sources, and a narrower wave for what the first missed. Each pulls out equations, concepts, real practice problems and how the claims connect. The content orchestrator writes it all up as one package, and a content review sends that back at most twice.

Phase 2 writes the plan. Each topic gets objectives and a teaching arc: the question it answers, the moves that get the student there, and what they should then be able to show. One medium decider, spawned once for the whole lesson so the media stay varied, ranks them for every topic. Then the plan goes to a person, as a dialog at a terminal or one message in a channel, and silence is not a yes. A build with no one watching records a hash of the plan and stops until an instruction quotes it, so an approval names the exact plan somebody read.

Building

Phase 3 builds every piece of media at once, and nothing a specialist makes reaches the lesson until it has been checked.

FIGURE 3 *one decision, then one specialist per item*



Rust is the decider, sage the four specialists, slate the staging area and ochre the session that assembles the lesson. The decider ran in phase 2; its plan is what phase 3 builds.

Main Claude spawns one specialist per item in the approved plan, all at once, each with that item's brief and the facts its topic needs. None of them writes into the lesson. Files the lesson serves, such as a rendered animation or an image, go into a staging area, and one script moves each into place only after checking that this run made it, so a killed or broken production leaves the old file untouched. Code is pasted in by main Claude, which also writes the prose itself, topic by topic, against the arc the plan approved. An update to an existing lesson builds on a branch in a git worktree of its own, so the author's checkout is never stashed or switched while a run is going.

Phase 3: `references/phase-3-execution.md`, *The run staging area* and *Parallel specialist spawning*. The worktree: `references/run-record.md`.

The agent team

Twelve agents ship in the skill's own folder, and the session running the build decides which model each one gets.

TABLE 1 *the twelve agents and where they work*

AGENT	PHASE	WHAT IT DOES
Research	1	Pulls facts out of one source, or researches a topic, with sources
Content orchestrator	1	Writes the workers' evidence up as one package
Content review	1, 4	Checks accuracy, then how each explanation unfolds
Medium decider	2	Picks the media for every topic, in one pass
Graphics	3	SVG graphs and matplotlib figures
Manim	3	Short animations, where motion carries the point
Interactive demo	3	Demos a student can drive, from shared parts
Web image	2, 3	Real photographs and spectra, licence checked
Code review	4	The lesson file: imports, maths, that it parses
Visual QA	4	One figure against a full presentation rubric
Scientific accuracy	4	Whether a figure shows what the science says
Interaction	4	Drives a running demo and tries every control

The session running the skill, main Claude in its own docs, spawns every agent itself, because a subagent cannot spawn another. Each worker writes its full output to a file and hands back a summary, so main Claude carries a paragraph per worker and whoever needs the detail reads the file.

Five of the twelve make judgements, about what the lesson teaches and whether it is right, and run on the session's own model. The other seven produce media or apply a rubric, and default to a smaller one. Main Claude can raise either for any spawn, and a lesson a student will sit an exam from runs its judgements on the strongest model there is. Nothing is lowered to save money unless the person asking said so.

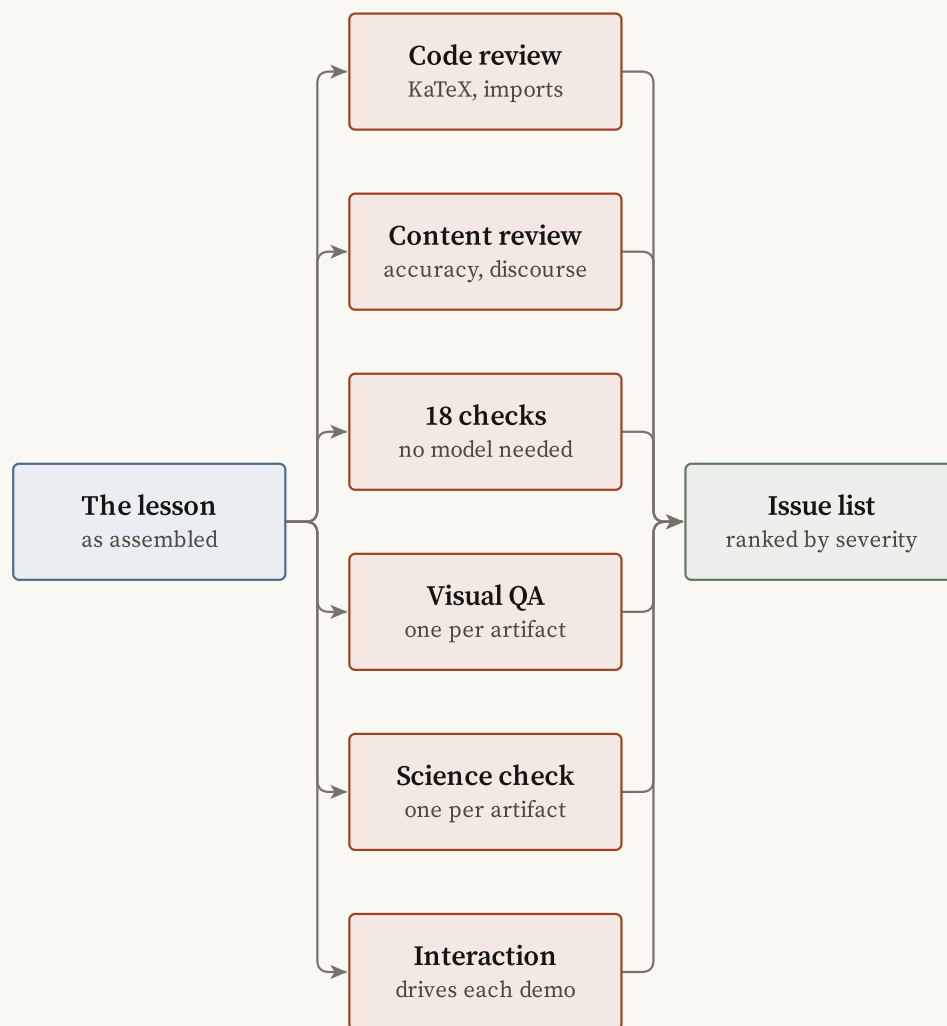
Table 1: SKILL.md, *Agent team*, and the description line of each file in *agents/*. The models: SKILL.md, *Model policy* and *Quality policy*, and the model: line of each agent. Who spawns what: *Agent team*.

Review, and the fix loop

Every reviewer runs at once, and none of them can edit.

Phase 4 fires every review in one batch. Each figure gets two reviewers that fail in different ways, one for how it looks and one for whether the science is right.

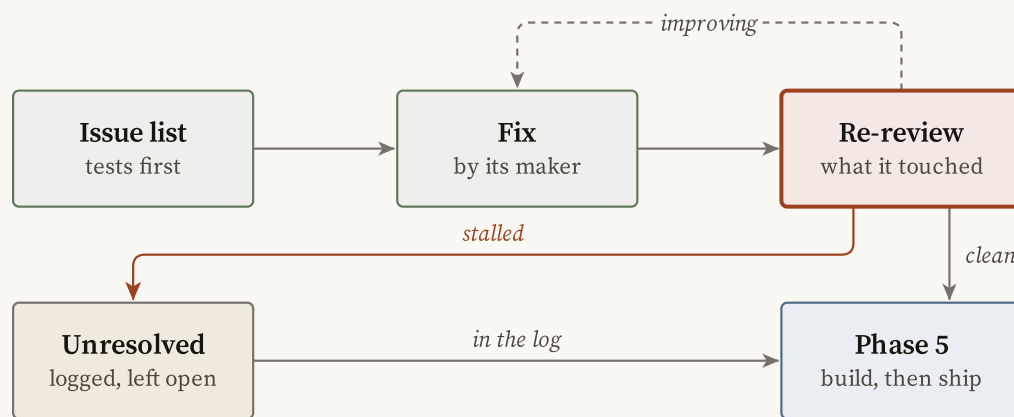
FIGURE 4 *phase 4 reviews everything in one batch*



Slate is the lesson, rust the six checks that can say no, sage the list they feed.

Every finding goes into one issue list, and into the run record before anything is fixed, so a crash in the loop loses nothing. A reviewer never fixes what it found, so a re-review is never a model grading its own work. A figure goes back to the specialist that made it, with the finding as its brief. Code and prose are fixed by main Claude, which assembled the file. Failures a test can prove, such as a parse error, a failing check or a broken build, are fixed first, because the rest are often their symptoms. After each round, the reviewers of whatever the fix touched run again.

FIGURE 5 *the loop that follows*



Sage is the work, rust the check the loop turns on, and slate the next phase. The dashed line is another round; the rust line is a thread given up on.

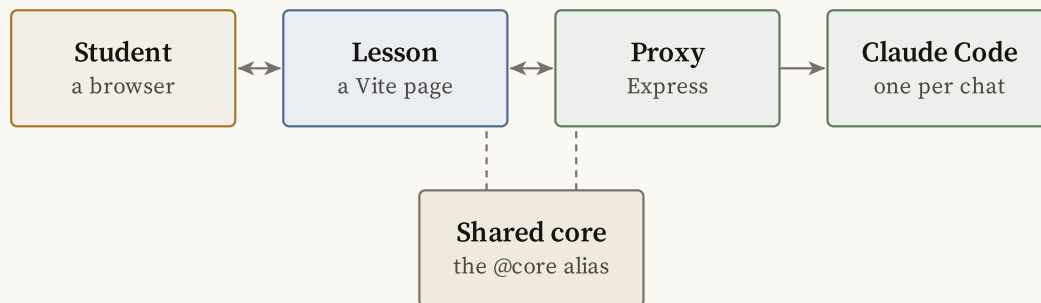
The loop is built to stop early rather than churn, because a loop that keeps rewriting risks breaking work that was already right. A thread is given up on when two rounds in a row fail to cut its open issues, or when the fixer twice reports no real progress, and the loop stops after six rounds whatever the numbers say. It is logged as unresolved and listed when the lesson ships. One finding stops the loop outright: a failure that shows the plan itself was wrong goes back to a person, not into another patch.

Phase 5 builds the lesson once more as a check, keeps course material out of the commit by default, builds a printed companion if the plan asked for one, and commits and pushes.

What a lesson is made of

Ten files of its own, and a shared core that does the rest.

FIGURE 6 *what runs when a student opens a lesson*



Ochre is the person, slate the page they read, sage the two processes behind it, and grey the code every lesson shares. Dashed lines are imports, not traffic.

What the pipeline ships is a Vite and React project, one per group of related topics, in a folder of its own under its course. It holds very little. The long file is the lesson's own text and figures; the page shell, the tutor chat, the maths renderer, the graph kit and the proxy all come from one shared core that every lesson imports. So a fix to the chat reaches every lesson at once, and a lesson stays readable as the thing it teaches.

The shell is one component of 654 lines. It lays out the page with a contents rail that follows the reader down, a column for the text, and a tutor panel that docks at the side, floats over the page, or opens in a window of its own. Maths is set with KaTeX, loaded with an eight second limit, and if it never arrives the reader sees the LaTeX source rather than a blank.

The layout and the shared core: `SKILL.md` and `references/bootstrap/_lesson-core/`. The proxy's routes: `_lesson-core/server/proxy.js`, lines 8 to 10. The shell: `_lesson-core/ui/LessonShell.jsx`.

TABLE 2 *the files a lesson keeps in its own folder*

FILE	WHAT IT DOES
src/<slug>.jsx	The lesson itself: its topics, its prose, and what the tutor is told about each topic
src/main.jsx	Five lines that mount it
index.html	The page it mounts into
package.json	Its own dependencies: React, Vite, KaTeX
server/proxy.js	One line that starts the shared proxy for this lesson
vite.config.js	Points the @core import at the shared core
test_lesson.cjs	Eighteen checks the lesson must pass before it ships
CLAUDE.md	What an agent editing this lesson needs to know
.gitignore	Keeps course material private by default
lesson_build.log.md	The build log, rendered from the run record

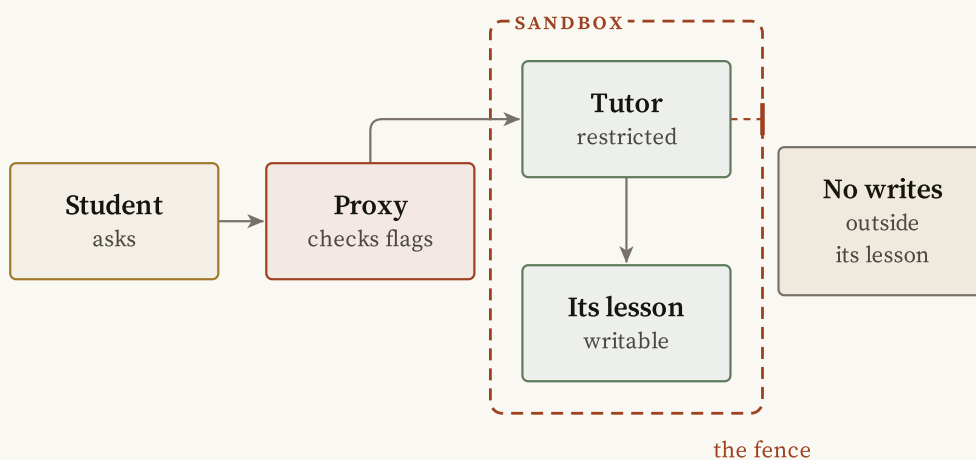
The tutor can do more than answer. It talks back to the page through a handful of tagged messages: one changes a graph the student is looking at, one draws a demo, one proposes an addition to the lesson that the student can accept. A student can Ctrl-click any part of the page to put it into the chat, and can open a thread on a paragraph, which starts as a fork of the main conversation and is folded back into it as a summary when the student is done. Whatever the model writes is rendered through an allowlist of elements, never as raw HTML.

A tutor that stays in its lesson

The tutor is a real Claude Code session, so the proxy decides what it can reach.

Each chat is its own Claude Code process, started by the lesson's proxy, so the tutor has real tools: it can read the course files, search the web, run commands and call on agents of its own.

FIGURE 7 *where the tutor may write*



Ochre is the student, rust the check that refuses, sage the tutor and the one folder it may write, and grey everything else. The rust dashed line is the fence.

Every tutor starts restricted. It loads no `CLAUDE.md`, no hooks and no user MCP servers, so it carries no one's instructions but the lesson's. It may edit and write only inside its own lesson, shell commands run in a sandbox that may write only there, and anything else it asks for is denied without a prompt. Before the proxy starts a single tutor it checks that the CLI offers every one of those options, and if one is missing, no tutor starts and every tutor route answers 503.

The agents a tutor may call are a team of its own: two that ship for tutoring, one that notices when a chat has filled a gap in the lesson and one that knows what the lesson already says, beside copies of the pipeline's agents for a graph, a check or a quick piece of research while the student waits. The tutor is their orchestrator, as main Claude is at build time.

The flags and the refusal: `_lesson-core/server/proxy.js`, *Tutor confinement*, and its fixture tests `/tutor-confinement/`. The tutor's team: `SKILL.md`, *Runtime tutor team*.

How it is checked

Two layers of tests, and nothing that can block a merge needs a model or a browser.

18

checks every lesson
passes before it ships

23

test scenarios in the
skill's own gate

12

runs left out, each
printed with its
command

73

commits since 13 April

The first layer is inside each lesson. Its eighteen checks parse the file, confirm every topic has what the tutor needs, and build the tutor's whole prompt to see that it fits the CLI's argument list. Past 28,000 characters the proxy has to pass it on standard input instead, where it loses its priority as instructions, so that one is a test and not a guideline.

The second layer is the skill's own gate, a script every pull request must pass before it can merge. It runs 23 self-contained scenarios, mostly in parallel, with a fake `claude` where one needs the CLI. A scenario that leaves a server behind fails the gate. The runs that need a browser, the real CLI or a model are not skipped in silence: the gate prints each one with the exact command that runs it.

A person decides what a lesson teaches. Everything after that has something that can say no.

THE ONE CLAIM THIS DOCUMENT MAKES

WHAT THE GATE DOES NOT JUDGE

Whether a lesson teaches well is judged by a reviewing model against a rubric. The evals that test that reviewer and the tutor on blind cases need a model too, so they are not in the gate, and a lesson can pass every test and still teach badly.

The eighteen checks: `references/checklists.md`. The gate, its fixtures and its exclusions: `tests/check.sh`. The ceiling: `_lesson-core/constants/promptBudget.js`. Teaching evals: `evals/teaching/README.md`. Commits: `git log`.

In use

It builds the lessons for a set of university courses, and most of what it has learned came from those lessons going wrong.

48	39	325
lessons, across 13 course folders	of them built before the current shell	commits to the lessons since 10 April

A built copy is published behind an access list, with the tutor live there since 7 September. A lesson can also have a printed companion, built by the skill that built this document.

TABLE 3 *what went wrong, and what changed*

WHAT HAPPENED	WHAT CHANGED
Cancelling a turn could kill the proxy itself	A process group is signalled only while the CLI still leads it
A dash in a long reply came out as a replacement character	The reader carries a character across a pipe-buffer boundary
A lesson built to the documented budget shipped a demoted prompt	The size is measured from the real prompt, and tested
A long reply was lost when the tab's stream dropped	The chat re-attaches to the turn still running

Each fix merged with a fixture in the gate that covers it. What is still open is shape: one app shell that mounts every lesson, typed actions in place of the tagged messages, and a tutor for the author kept apart from the student's.